

Databricks Lakehouse Interview & Practical Guide

Questions, Solutions, Code Snippets, Delta Live Tables Expectations, & Unity Catalog Cheat Sheet

Target Role:	Databricks Data Engineer / Lakehouse Architect
Candidate	5+ YOE Apache Spark, PySpark, Delta Lake, DLT Expectations, Unity Catalog, Workflow
Background:	Orchestration
Document Version:	v1 (Databricks Platform Edition - Delta Lake & DLT Focus)

SECTION 1: DELTA LAKE ARCHITECTURE, PERFORMANCE & GOVERNANCE

1. How do Delta Lake OPTIMIZE, Z-Ordering, and Liquid Clustering improve performance over standard Parquet files?

Technical Deep-Dive

Candidate context: Leverages PySpark and Spark SQL extensively across enterprise datalakes.

Key Talking Points & Description:

- **Small File Problem:** Compacts small files into ~1GB parquet files using OPTIMIZE.
- **Z-Ordering & Liquid Clustering:** Co-locates related data across multiple dimensions for optimal file skipping without rigid partitioning constraints.
- **Time Travel & Vacuum:** Queries past states via Delta transaction logs and cleans obsolete parquet files with VACUUM.

Reference DDL / SQL Snippet:

```
-- Optimize and apply Z-Ordering on Delta Table
OPTIMIZE main.finance.transactions
ZORDER BY (customer_id, store_id);

-- Alternatively, using Liquid Clustering (Databricks Modern Standard)
CREATE TABLE main.finance.transactions_clustered (
  transaction_id STRING,
  customer_id STRING,
  transaction_date DATE,
  amount DECIMAL(10,2)
)
USING DELTA
CLUSTER BY (transaction_date, customer_id);
```

2. How do you implement fine-grained governance (Row-Level Security and Column Masking) using Unity Catalog?

Governance & Security

Candidate context: Expertise in enterprise data governance, access controls, and compliance.

Key Talking Points & Description:

- **Unity Catalog 3-Level Namespace:** catalog.schema.table model.
- **Fine-Grained Controls:** Uses SQL UDF-based row filters and column masking functions bound directly to Unity Catalog tables.

Reference Unity Catalog Governance Snippet:

```
-- 1. Create Column Masking UDF for SSN
CREATE FUNCTION main.security.ssn_mask(ssn STRING)
RETURN IF(IS_ACCOUNT_GROUP_MEMBER('hr_admin'), ssn, CONCAT('***-**- ', RIGHT(ssn, 4)));

-- 2. Apply Masking Function to Table Column
ALTER TABLE main.finance.customer_pii
ALTER COLUMN ssn SET MASK main.security.ssn_mask;

-- 3. Create Row Filter Function
CREATE FUNCTION main.security.region_filter(region STRING)
RETURN IS_ACCOUNT_GROUP_MEMBER('us_execs') OR region = 'NORTH_AMERICA';

-- 4. Apply Row Filter Policy
ALTER TABLE main.finance.sales_records
SET ROW FILTER main.security.region_filter ON (region);
```

3. How do you enforce Data Quality Expectations and handling rules in Delta Live Tables (DLT)?

Data Quality & DLT

Candidate context: Background in operationalizing data quality expectations across Medallion architecture (Bronze -> Silver -> Gold).

Key Talking Points & Description:

- **DLT Expectations:** Declarative rules defining quality constraints using decorators or SQL properties.
- **Constraint Actions:** EXPECT (logs metrics), EXPECT OR DROP (drops bad records), and EXPECT OR FAIL (halts pipeline).

Reference Python Delta Live Tables (DLT) Snippet:

```
import dlt
from pyspark.sql.functions import col

# Bronze Layer: Ingest Raw Data
@dlt.table(comment="Raw streaming ingest from Cloud Storage")
def bronze_transactions():
    return spark.readStream.format("cloudFiles") \
        .option("cloudFiles.format", "json") \
        .load("/mnt/raw_data/transactions/")

# Silver Layer: Apply Expectations & Data Quality Constraints
@dlt.table(comment="Cleaned transactions with quality enforcement")
@dlt.expect("valid_transaction_id", "transaction_id IS NOT NULL")
@dlt.expect_or_drop("valid_amount", "amount > 0")
@dlt.expect_or_fail("valid_customer", "customer_id IS NOT NULL")
def silver_transactions():
    return dlt.read_stream("bronze_transactions") \
        .select("transaction_id", "customer_id", "amount", "transaction_date")
```

4. How do you handle severe data skew during large-scale PySpark join operations in Databricks?

Performance & Optimization

Candidate context: Experience resolving Spark memory issues, spills to disk, and executor OOM errors.

Key Talking Points & Description:

- **Adaptive Query Execution (AQE):** Databricks AQE automatically detects skew and splits skewed partitions into smaller sub-partitions.
- **Skew Hinting / Salting:** Explicitly adding `/*+ SKEW('table_name', 'col_name') */` or programmatically salting join keys.

Reference PySpark Skew Optimization Snippet:

```
# Enable AQE Skew Join Handling in Databricks
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")

# PySpark Skew Hint explicit declaration
large_df = spark.table("main.finance.large_transactions").hint("skew", "customer_id")
dim_df = spark.table("main.finance.customer_dim")

joined_df = large_df.join(dim_df, on="customer_id", how="inner")
joined_df.write.mode("overwrite").saveAsTable("main.finance.analytics_summary")
```

SECTION 2: PIPELINE INGESTION & ORCHESTRATION

5. How does Auto Loader work for incremental file ingestion into Delta Lake, and why is it preferred over traditional streaming? Ingestion Engine

Candidate context: Building fault-tolerant, scalable file processing pipelines.

Key Talking Points & Description:

- **CloudFiles Source:** Uses file notification services (AWS SQS, Azure Event Grid, GCP PubSub) or directory listing to efficiently discover new files.
- **Schema Evolution & Inference:** Automatically detects new columns and safely writes unexpected data to a rescue column (`_rescued_data`).

Reference Auto Loader PySpark Snippet:

```
# Auto Loader ingestion pattern with schema inference and evolution
df = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("cloudFiles.schemaLocation", "gs://my-lakehouse/checkpoints/schema_store")
    .option("cloudFiles.inferColumnTypes", "true")
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    .load("gs://my-lakehouse/raw_landing/sales/"))

(df.writeStream
    .format("delta")
    .option("checkpointLocation", "gs://my-lakehouse/checkpoints/sales_write")
    .outputMode("append")
    .toTable("main.finance.silver_sales"))
```

6. How do you structure multi-task workflows and dependency graphs in Databricks Workflows?

Orchestration

Candidate context: Enterprise orchestration using native Databricks Workflows and job clusters.

Key Talking Points & Description:

- **Databricks Workflows:** Orchestrates tasks (Notebooks, DLT pipelines, Python scripts, SQL queries).
- **Cost Efficiency:** Uses shared or ephemeral single-job clusters to run production DAGs with minimal DBU usage.

Reference Databricks Asset Bundles (DABs) / Job Definition JSON Snippet:

```
{
  "name": "Daily_Medallion_ETL_Job",
  "email_notifications": { "on_failure": ["dataops@company.com"] },
  "tasks": [
    {
      "task_key": "ingest_autoloader",
      "notebook_task": { "notebook_path": "/Repos/prod/ingest_raw" },
      "job_cluster_key": "etl_cluster"
    },
    {
      "task_key": "run_dlt_quality_pipeline",
      "depends_on": [ { "task_key": "ingest_autoloader" } ],
      "pipeline_task": { "pipeline_id": "a1b2c3d4-5678-90ab-cdef-1234567890ab" }
    }
  ]
}
```

SECTION 3: DATABRICKS LAKEHOUSE CHEAT SHEET

Component	Primary Purpose	Key Technical Highlights & Snippets
Delta Lake	Storage Layer / ACID Transactions	Brings ACID compliance, schema enforcement, time travel, OPTIMIZE, and Liquid Clustering (CLUSTER BY) to cloud object stores.
Unity Catalog	Centralized Data Governance	Unified metadata, fine-grained access control, column masking functions (SET MASK), row filters, and automated lineage.
Delta Live Tables (DLT)	Declarative Data Engineering Framework	Automates ETL processing with built-in expectations (@dlt.expect, @dlt.expect_or_drop, @dlt.expect_or_fail) across Medallion layers.
Auto Loader	Scalable File Ingestion Engine	Efficient incremental processing of incoming cloud files (GCS/S3/ADLS) using cloudFiles format and rescue column schema tracking.
Photon Engine	C++ Native Query Vectorization	High-performance native C++ execution engine that speeds up Spark SQL and DataFrames queries while reducing total compute time.
Databricks Workflows	Job Orchestration Engine	Native workflow DAG orchestrator running notebooks, Python files, and DLT jobs on ephemeral job clusters for optimal DBU management.